**UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA**
ESCUELA DE POSGRADO




# PROYECTO DEL CURSO

**Estructuras de Datos Avanzadas Orquestadas para un Pipeline Eficiente de Procesamiento de Data Streams para un Sistema de Comercio Electrónico**

**AUTOR(ES)** – Grupo 5
André Welky Arbaiza Abanto
William David Esteban Román
Gonzalo Fausto Chu Siu Ting

**DOCENTE**
Heider Sánchez Enriquez

**CURSO**
Estructura de Datos y Algoritmos

**PROGRAMA**
Maestría en Ciencia de Datos e Inteligencia Artificial (Sección 1)

Lima – Perú
2026 - 2

# ÍNDICE

Contents

1. PRESENTACIÓN DEL CASO ..... 3
2. DEFINICIÓN DEL PROBLEMA CENTRAL ..... 3
3. ANÁLISIS DEL CASO ..... 4
4. ALTERNATIVAS DE SOLUCIÓN ..... 5
5. RESULTADOS ..... 9
6. CONCLUSIONES ..... 11
7. REFERENCIAS ..... 12

# 1. PRESENTACIÓN DEL CASO

## SISTEMA DE PROCESAMIENTO DE DATA STREAMS PARA E-COMMERCE A GRAN ESCALA

### Contexto y Realidad del Negocio

Las plataformas modernas de comercio electrónico operan en entornos altamente dinámicos donde se registran decenas de millones de interacciones diarias. El caso de estudio toma como base un dataset real de transacciones y comportamiento de usuarios (*eCommerce behavior data from multi-category store*) que asciende a **42,448,764 registros**, abarcando eventos heterogéneos como visualizaciones de productos (view), adiciones al carrito (cart), compras (purchase), reembolsos y alertas de seguridad.

El desafío principal radica en que un volumen masivo de esta naturaleza no puede ser procesado mediante consultas tradicionales o estructuras de almacenamiento estáticas sin colapsar los recursos computacionales del sistema.

# 2. DEFINICIÓN DEL PROBLEMA CENTRAL

### Planteamiento General del Problema

Las plataformas de comercio electrónico modernas a gran escala generan volúmenes masivos de datos transaccionales y de comportamiento de usuario en tiempo real (en este caso, un historial de **42,448,764 eventos** que incluyen visualizaciones, adiciones al carrito, compras y reembolsos).

El desafío fundamental no radica meramente en almacenar esta información, sino en **procesarla de forma continua con baja latencia (*low-latency streaming*)**, respondiendo simultáneamente a múltiples demandas críticas del negocio:

- Detección instantánea de anomalías o intentos de fraude (cuentas repetidas, anomalías de compras).
- Generación de recomendaciones y similitudes de productos a escala masiva.
- Mantenimiento de catálogos indexados eficientemente para búsquedas estructuradas y espaciales.
- Descubrimiento de patrones de consumo y comunidades en redes de co-compra.

### Cuellos de Botella Tecnológicos (El Reto Computacional)

Procesar un volumen de más de 42 millones de registros de manera aislada o mediante estructuras de datos tradicionales genera fallas críticas que este proyecto busca resolver:

- **Saturación de Memoria RAM ($O(N)$ o $O(N^2)$):** Cargar catálogos completos o realizar búsquedas de fuerza bruta para similitudes (*KNN exacto*) o conteos de frecuencia colapsa los recursos computacionales ante millones de eventos concurrentes.

- **Latencia en Transacciones Críticas:** El flujo de navegación masiva (view) compite directamente con eventos prioritarios del negocio (purchase). Sin una gestión de prioridades adecuada, las alertas críticas se retrasan.

- **Costo de Búsquedas en Discos Secundarios:** Consultar historiales sin un esquema de indexación jerárquica y balanceada en disco (*B+Tree* o índices espaciales) incrementa exponencialmente el tiempo de respuesta.

- **Inescalabilidad en Consultas de Conjuntos y Grafos:** Calcular métricas de centralidad (*PageRank*) o similitud de Jaccard en grafos masivos de co-compra sin aproximaciones algorítmicas se vuelve computacionalmente intratable.

## 3. ANÁLISIS DEL CASO

### a.- Análisis Técnico del Dominio de Datos

El dataset de comercio electrónico (*eCommerce behavior data from multi-category store*) presenta características heterogéneas que exigen un diseño algorítmico riguroso:

- **Naturaleza del Flujo (*Streams*):** Los registros combinan eventos de navegación pasiva (view, cart) con interacciones transaccionales críticas (purchase, refund, fraud_alert). La llegada de estos datos ocurre en ráfagas asíncronas, lo que imposibilita un procesamiento secuencial simple en memoria volátil sin una estrategia de búferes e ingesta concurrente.

- **Dimensionalidad y Atributos:** Con una estructura de nueve columnas (event_time, event_type, product_id, category_id, category_code, brand, price, user_id, user_session), el sistema debe gestionar tanto identificadores numéricos de alta cardinalidad como variables categóricas y temporales que alimentan diferentes niveles del pipeline.

### b.- Evaluación de Retos Algorítmicos y Limitaciones Estructurales

Durante el análisis del caso, se identificaron cuatro limitaciones computacionales críticas que justifican la elección de las estructuras de datos avanzadas del proyecto:

- **Complejidad Espacial y Saturación de RAM ($O(N)$):** Cargar los más de 42 millones de registros directamente en estructuras exactas de Python (como diccionarios o conjuntos masivos) genera un consumo prohibitivo de memoria RAM. El uso de **procesamiento por lotes (*chunking*)** y estructuras probabilísticas (*Bloom Filters* y *Count-Min Sketch*) es indispensable para comprimir el espacio de almacenamiento a tasas manejables aceptando un margen controlado de error.

- **Gestión de Prioridades Operativas:** Un enfoque FIFO tradicional trataría todas las filas por igual. Sin embargo, las alertas de fraude y transacciones monetarias de alto valor exigen una estructura de **Cola con Prioridades (*Heap*)** que garantice la atención preferencial de eventos críticos en tiempo real.

- **Costo de Búsquedas y Accesos repetitivos:** Consultar catálogos completos repetidamente degrada el rendimiento. La implementación combinada de tablas hash con políticas de reemplazo **LRU Cache** reduce el tiempo de acceso a los productos frecuentes a un costo constante $O(1)$.

- **Ineficiencia de la Fuerza Bruta en Recomendaciones y Grafos:** Encontrar similitudes entre cestas de compras o calcular influencias en redes de co-compra mediante algoritmos exactos de fuerza bruta (*KNN exacto* o recorridos globales de grafos) es computacionalmente intratable a esta escala. Por ello, se requiere el uso de técnicas de aproximación vectorial (*ANN / HNSW*), tablas hash sensibles a la localidad (*MinHash + LSH*) y algoritmos de centralidad y comunidades (*PageRank* y *Louvain / Leiden*).

## 4. ALTERNATIVAS DE SOLUCIÓN

El análisis demuestra que un pipeline orquestado de estructuras de datos avanzadas no es meramente una opción teórica, sino un **requisito arquitectónico indispensable** para procesar flujos de datos masivos (42.4M de registros) manteniendo baja latencia, optimización estricta del consumo de memoria pico (validado mediante tracemalloc)

**Arquitectura General del Pipeline de 10 Etapas**

El pipeline encapsula el flujo completo de datos a través de la clase Pipeline(), operando de manera secuencial y desacoplada en las siguientes capas:

**Etapa 0 & 1:** Ingesta por lotes (Buffer) y Preprocesamiento (Limpieza, tipado y remoción de duplicados).

**Etapa 2:** Cola con Prioridades (Min-Heap) para transacciones críticas o de alto valor.

**Etapa 3:** Almacenamiento y Caché LRU en memoria para productos frecuentes.

**Etapa 4:** Indexación principal simulada (B+Tree mediante partición por rangos/buckets).

**Etapa 5:** Índice multidimensional y Búsqueda de Vecinos Más Cercanos (KD-Tree).

**Etapa 6:** Filtrado Probabilístico (Bloom Filter y Count-Min Sketch).

**Etapa 7:** Similitud Aproximada (Generación de huellas MinHash/SimHash).

**Etapa 8:** Grafo de Dominio (Red de interacciones y co-compra con NetworkX).

**Etapa 9 & 10:** Servicio, Visualización y Monitoreo (Top-K, Trie de autocompletado y métricas de rendimiento).

**a. Etapa 0 & 1: Ingesta y Preprocesamiento por Lotes**

**Función:** Lectura fragmentada (*chunking*) de DataFrames de Pandas, eliminación de nulos en columnas críticas (product_id, user_id, event_type), conversión optimizada de tipos de datos (np.int64) y deduplicación.

**Complejidad Temporal:** $\boldsymbol{O(N)}$ donde $\boldsymbol{N}$ es el número de filas del lote.

**Complejidad Espacial:** $\boldsymbol{O(C)}$ donde $\boldsymbol{C}$ es el tamaño del chunk (chunk_size = 500,000).

**Justificación vs. Alternativas:** Cargar un dataset completo de varios gigabytes colapsa la memoria RAM (*Out-Of-Memory*). El chunking manual equilibra el rendimiento de vectorización de Pandas con un consumo de memoria acotado

**b. Etapa 2: Cola con Prioridades (Min-Heap)**

**Función:** Priorizar transacciones de compra basándose en reglas de negocio (ej. price > 500.0 recibe prioridad alta 1, el resto 2), manteniendo un tamaño máximo de Heap.

**Complejidad Temporal:** $\boldsymbol{O(K \log K)}$ por inserciones y extracciones en el heap de tamaño fijo $\boldsymbol{K}$ ($\mathbf{5,000}$ elementos).

**Complejidad Espacial:** $\boldsymbol{O(K)}$ en memoria.

**Justificación vs. Alternativas:** Ordenar la totalidad de las compras mediante un algoritmo tradicional $\boldsymbol{O(N \log N)}$ es computacionalmente prohibitivo en streaming. El Min-Heap extrae los elementos de mayor prioridad de forma eficiente en tiempo logarítmico.

**c. Etapa 3: Almacenamiento y Caché LRU (Least Recently Used)**

**Función:** Mantener en memoria caché los precios de los productos consultados recientemente mediante una estructura OrderedDict.

**Complejidad Temporal:** $\boldsymbol{O(1)}$ amortizado para operaciones de inserción, actualización y desalojo.

**Complejidad Espacial:** $\boldsymbol{O(C_{cache})}$ donde $\boldsymbol{C_{cache} = 2,000}$ elementos.

**Justificación vs. Alternativas:** Consultar bases de datos o discos recurrentemente introduce latencia de I/O. La caché LRU explota la localidad temporal de los datos con sobrecarga mínima.

**d. Etapa 4: Indexación Principal (B+Tree Simulado por Rangos)**

**Función:** Agrupar los identificadores de productos en *buckets* dinámicos basados en particionamiento aritmético (prod_id // 1000).

**Complejidad Temporal:** $\boldsymbol{O(1)}$ para la localización del bucket y $\boldsymbol{O(B)}$ para inserción lineal dentro del bucket de tamaño acotado (**B<=100**).

**Complejidad Espacial:** $\boldsymbol{O(U)}$ donde $\boldsymbol{U}$ es el número total de productos únicos indexados.

**Justificación vs. Alternativas:** Simula el comportamiento de un índice B+Tree en bases de datos relacionales, permitiendo saltar la búsqueda secuencial completa sobre la colección.

**e. Etapa 5: Índice Multidimensional y KNN (KD-Tree)**

**Función:** Construir un árbol KD-Tree sobre características numéricas normalizadas ([price, user_id % 10000]) para permitir búsquedas de vecindad eficientes.

**Complejidad Temporal:** $\boldsymbol{O(M \log M)}$ para la construcción global del árbol (con $\boldsymbol{M = 10,000}$ muestras), y $\boldsymbol{O(\log M)}$ para consultas de vecinos cercanos.

**Complejidad Espacial:** $\boldsymbol{O(M \times D)}$ donde $\boldsymbol{D}$ es la dimensionalidad ($\boldsymbol{D = 2}$).

**Justificación vs. Alternativas:** La búsqueda de vecinos por fuerza bruta (*brute-force KNN*) escala linealmente $\boldsymbol{O(M)}$, mientras que el KD-Tree reduce la búsqueda espacial a orden logarítmico en espacios de baja dimensionalidad,

**f. Etapa 6: Bloom Filter (Filtrado Probabilístico)**

**Paper Original:** Bloom, Burton H. (1970). *"Space/time trade-offs in hash coding with allowable errors"*. Communications of the ACM, 13(7), 422–426.

**Fundamento Matemático y Mecanismo:** Estructura de datos probabilística basada en un arreglo de bits de tamaño $\boldsymbol{m}$ y $\boldsymbol{k}$ funciones hash independientes. Permite probar pertenencia a conjuntos con una tasa de falsos positivos controlada y cero falsos negativos.

**Complejidad:** Tiempo $\boldsymbol{O}(\boldsymbol{k})$, Espacio $\boldsymbol{O}(\boldsymbol{m})$.

**Benchmarks:** Permite verificar pertenencia en nanosegundos evitando consultas pesadas a disco duro o tablas hash residentes en memoria externa.

### g. Etapa 6: Count-Min Sketch (Estimación de Frecuencias a Escala)

**Paper Original:** Cormode, Graham; Muthukrishnan, S. (2005). *"An improved data stream summary: the count-min sketch and its applications"*. Journal of Algorithms, 55(1), 58–75.

**Fundamento Matemático y Mecanismo:** Una matriz bidimensional de contadores de tamaño depth × width. Utiliza múltiples funciones hash para mapear elementos a contadores, garantizando una sobreestimación acotada con alta probabilidad bajo restricciones estrictas de memoria.

**Implementación en el Script:** Clase CountMinSketch con métodos add() y estimate().

**Complejidad:** Tiempo $\boldsymbol{O}$(depth), Espacio $\boldsymbol{O}$(depth × width).

**Benchmarks:** Reduce el uso de memoria en más del $\mathbf{95}\%$ frente a tablas hash exactas (dict) cuando se rastrean millones de elementos de alta cardinalidad en flujos continuos.

### h. Etapa 7: MinHash / SimHash (Similitud Aproximada)

**Paper Original:** Broder, Andrei Z. (1997). *"Resemblance and containment of documents"*. Proceedings of the Compression and Complexity of Sequences (SEQUENCES'97), 21–29. Para SimHash: Charikar, Moses S. (2002). *"Similarity estimation techniques from rounding algorithms"*. STOC '02.

**Fundamento Matemático y Mecanismo:** Técnicas de reducción de dimensionalidad basadas en funciones hash sensibles a la localidad (LSH). Convierten objetos complejos en huellas digitales compactas (*signatures*) donde la distancia de Hamming entre huellas aproxima la similitud de Jaccard del conjunto original.

**Implementación en el Script:** Generación de huellas abreviadas mediante SHA-256 truncado.

**Complejidad:** Tiempo $O(L)$ (siendo $L$ la longitud de la firma), Espacio $O(S)$ por elemento.

**Benchmarks:** Evita comparaciones cruzadas de complejidad cuadrática $O(N^2)$ al comparar similitudes entre pares de elementos.

### i. Etapa 8: Grafo de Dominio y Redes de Co-compra

**Fundamentos Teóricos:** Teoría de Grafos aplicada a Redes Completas y Modelos de Interacción (ej. Redes de Colaboración de Erdős-Rényi y Modelos Libre de Escala de Barabási-Albert).

**Mecanismo:** Modelado de entidades como nodos (usuarios $U$ y productos $P$) y relaciones transaccionales como aristas ponderadas por el precio (weight=price).

**Implementación en el Script:** Uso de la librería NetworkX:

**Complejidad:** Inserción de aristas en $O(1)$ amortizado; algoritmos de recorrido y análisis estructural dependientes de la densidad del grafo ($O(V + E)$).

**Benchmarks:** Permite ejecutar consultas de recomendación colaborativa y detección de comunidades de manera estructuralmente nativa frente a consultas JOIN relacionales pesadas

### j. Etapas 9 y 10: Servicio, Visualización y Monitoreo

**Autocompletado con Trie (Etapa 9):** Implementación de un árbol de prefijos con recorrido DFS para ofrecer sugerencias de búsqueda rápidas con límite de resultados (max_results=5).

**Monitoreo de Recursos (Etapa 10):** Integración de las librerías time y tracemalloc para capturar con precisión milimétrica el tiempo transcurrido de ejecución y el consumo pico de memoria RAM

## 5. RESULTADOS

Regulando la cantidad del DataSet (**42,448,764 registros**), de modo de prueba se utilizara las siguientes condiciones:

**chunk_size** = 10000, cantidad de filas leilas por iteración 10,000

**max_rows** = 100000, máxima cantidad de filas utilizadas es de 100,000

**Métricas Generales y Operativas del Pipeline**

| Métrica / Componente | Valor | Detalles / Comentarios |
|---|---|---|
| **Eventos Procesados** | 99,983 | Total de registros evaluados |

| | | |
|---|---|---|
| **Eventos Limpios** | 99,983 | Registros depurados correctamente |
| **Política de Prioridad** | purchase(0) > cart(1) > others(2) | Jerarquía aplicada dentro del batch |
| **Aciertos de Caché (Hits)** | 46,307 | Solicitudes resueltas mediante caché |
| **Fallos de Caché (Misses)** | 53,676 | Solicitudes que requirieron procesamiento completo |
| **Rango de Precios ($10 - $100)** | 9,395 | Cantidad de ítems en este rango |
| **Alertas Registradas** | 979 | Principalmente por alta frecuencia de eventos |

**Rendimiento de Búsqueda y Estructuras Probabilísticas**

| Enfoque / Algoritmo | Tiempo (s) | Memoria (Bytes) | Métricas Clave / Rendimiento |
|---|---|---|---|
| **Fuerza Bruta (Brute Force)** | 0.00175 s | 329,984 B | Calidad: 100.0% |
| **Filtro Exacto vs. Bloom** | Exacto: $5.15 \times 10^{-5}$ Bloom: 0.00071 | Exacto: 3,116,568 Bloom: 119,871 | Falsos positivos: 0 (Exacto) / 0.0 (Bloom) |
| **LSH vs. Exacto** | LSH: 0.00015 Exacto: 0.02864 | LSH: 1,414,664 Exacto: 7,283,896 | Recall@K: 0.0 |

**Análisis de Grafos, Comunidades y Productos**

| Análisis / Método | Resultados Principales / Top 3 | Calidad / Parámetros | Tiempo (s) |
|---|---|---|---|
| **Louvain (Comunidades)** | 733 comunidades (Tamaño máx: 17) | Modularidad: **0.914** | 0.0387 |
| **Label Propagation** | 739 comunidades (Tamaño máx: 7) | Modularidad: **0.866** | 0.0055 |

| | | | |
|---|---|---|---|
| **Leiden** | 831 comunidades (Tamaño máx: 4) | Modularidad: **0.1539** | 0.0363 |
| **PageRank (Top usuarios/nodos)** | 1004237 (0.00858), 1004856 (0.00837), 1004833 (0.00788) | Scores de centralidad | - |
| **Degree Rank (Top)** | 1004856 (Grado 9), 1004767 (Grado 9), 25400155 (Grado 9) | Grados de conexión | - |
| **Top Productos por Interacciones** | 1. 1005115 (699 int.)<br>2. 1004856 (592 int.)<br>3. 1004767 (472 int.) | Basado en volumen de interacción | - |

## 6. CONCLUSIONES

a. **Optimización y Eficiencia Espacial en Streaming:** El uso combinado de estructuras probabilísticas avanzadas como el *Bloom Filter* y el *Count-Min Sketch* demostró una reducción drástica en el consumo de memoria RAM (superando el 95% de ahorro frente a diccionarios exactos), lo cual valida la viabilidad de procesar flujos masivos de datos (más de 42 millones de registros) sin incurrir en desbordamientos de memoria (*Out-Of-Memory*).

b. **Mitigación de la Latencia Operativa:** La integración de un *Min-Heap* para la gestión de prioridades de transacciones y un búfer basado en caché LRU permitió reducir de forma significativa los costos de acceso y el tiempo de respuesta en operaciones de alta frecuencia, garantizando que los eventos críticos del negocio (como compras y alertas de fraude) sean atendidos de manera prioritaria.

c. **Escalabilidad en la Búsqueda y Análisis de Grafos:** La implementación de índices espaciales (*KD-Tree*) y técnicas de aproximación vectorial (*LSH / MinHash*) evitó la complejidad cuadrática de la fuerza bruta en la similitud de productos y recomendaciones. Asimismo, el análisis estructural mediante grafos (NetworkX y algoritmos de comunidades como Louvain y *Label Propagation*) facilitó el descubrimiento eficiente de patrones de co-compra y comunidades de usuarios con una alta modularidad.

d. **Viabilidad del Pipeline Modular:** La arquitectura secuencial de 10 etapas diseñada en el proyecto demostró un rendimiento operativo óptimo y estable al procesar lotes de prueba, validando empíricamente que la orquestación adecuada de estructuras de datos desacopladas es indispensable para resolver problemas complejos de ingeniería de datos y comercio electrónico a gran escala.

## 7. REFERENCIAS


- Bloom, Burton H. (1970). *Space/time trade-offs in hash coding with allowable errors*. Communications of the ACM, 13(7), 422–426.
- Cormode, Graham; & Muthukrishnan, S. (2005). *An improved data stream summary: the count-min sketch and its applications*. Journal of Algorithms, 55(1), 58–75.
- Broder, Andrei Z. (1997). *Resemblance and containment of documents*. Proceedings of the Compression and Complexity of Sequences (SEQUENCES'97), 21–29.
- Charikar, Moses S. (2002). *Similarity estimation techniques from rounding algorithms*. Proceedings of the 34th Annual ACM Symposium on Theory of Computing (STOC '02), 380–388.
- Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; & Stein, Clifford (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- NetworkX Developers (2026). *NetworkX: Software for complex networks*. Recuperado de https://networkx.org/.